Integrating Rust Bulletproofs with Python using PyO3

Zero-Knowledge Proof Implementation Guide

March 6, 2026

1 Introduction

This document outlines the step-by-step process of integrating the Rust-based `bulletproofs` cryptographic library into a Python environment. By utilizing `PyO3` and `Maturin`, we can compile high-performance Rust cryptography directly into a native Python module, bypassing the need for unsafe C-bindings.

2 Environment Setup

To compile Rust code into a Python environment, both the Rust compiler and the Maturin build system are required.

1. **Install Rust:** Download and run the installer from `https://rustup.rs/`. On Windows, ensure the Visual Studio C++ build tools are installed.
2. **Install Maturin:** Activate your target Python environment (e.g., Anaconda) and install the Maturin package:

   ```
   pip install maturin
   ```

3 Project Initialization

Navigate to your working directory in the terminal and initialize the Rust-Python bridge project:

```
maturin new python_bulletproofs
```

When prompted for the binding type, select `pyo3`. Navigate into the newly created directory:

```
cd python_bulletproofs
```

4 Dependency Configuration

The `Cargo.toml` file manages Rust dependencies. To ensure compatibility with the latest Dalek cryptographic suite, update the file to use the next-generation (`-ng`) crates. Replace the contents of `Cargo.toml` with:

```
[package]
name = "python_bulletproofs"
version = "0.1.0"
edition = "2021"
```

```
[lib]
name = "python_bulletproofs"
crate-type = ["cdylib"]

[dependencies]
pyo3 = { version = "0.19.0", features = ["extension-module"] }
bulletproofs = "4.0.0"
curve25519-dalek-ng = "4.1"
merlin = "3.0"
rand = "0.8"
```

5 Rust Bridge Implementation

The bridge logic resides in `src/lib.rs`. This file exposes two functions to Python: `prove_range` and `verify_range`. Replace the contents of `src/lib.rs` with the following code:

```
use pyo3::prelude::*;
use pyo3::types::PyBytes;
use bulletproofs::{BulletproofGens, PedersenGens, RangeProof};
use curve25519_dalek_ng::scalar::Scalar;
use curve25519_dalek_ng::ristretto::CompressedRistretto;
use merlin::Transcript;
use rand::rngs::OsRng;

#[pyfunction]
fn prove_range(
    py: Python,
    secret_value: u64,
    bit_length: usize
) -> PyResult<(&PyBytes, &PyBytes)> {
    let pc_gens = PedersenGens::default();
    let bp_gens = BulletproofGens::new(64, 1);

    let blinding_factor = Scalar::random(&mut OsRng);
    let mut prover_transcript = Transcript::new(b"FaceZKRangeProof");

    let (proof, commitment) = RangeProof::prove_single(
        &bp_gens,
        &pc_gens,
        &mut prover_transcript,
        secret_value,
        &blinding_factor,
        bit_length,
    ).expect("Failed to generate range proof");

    let proof_bytes = proof.to_bytes();
    let comm_bytes = commitment.to_bytes();

    Ok((
        PyBytes::new(py, &comm_bytes),
```

```
        PyBytes::new(py, &proof_bytes)
    ))
}

#[pyfunction]
fn verify_range(
    commitment_bytes: &[u8],
    proof_bytes: &[u8],
    bit_length: usize
) -> PyResult<bool> {
    let pc_gens = PedersenGens::default();
    let bp_gens = BulletproofGens::new(64, 1);

    if commitment_bytes.len() != 32 {
        return Ok(false);
    }
    let mut comm_arr = [0u8; 32];
    comm_arr.copy_from_slice(commitment_bytes);
    let commitment = CompressedRistretto(comm_arr);

    let proof = match RangeProof::from_bytes(proof_bytes) {
        Ok(p) => p,
        Err(_) => return Ok(false),
    };

    let mut verifier_transcript = Transcript::new(b"FaceZKRangeProof");
    let is_valid = proof.verify_single(
        &bp_gens,
        &pc_gens,
        &mut verifier_transcript,
        &commitment,
        bit_length,
    ).is_ok();

    Ok(is_valid)
}

#[pymodule]
fn python_bulletproofs(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(prove_range, m)?)?;
    m.add_function(wrap_pyfunction!(verify_range, m)?)?;
    Ok(())
}
```

6 Compilation

To compile the Rust code and install it directly into the active Python environment, run the following command in the terminal from the root of the `python_bulletproofs` directory:

```
maturin develop --release
```

7 Mathematical Shift for Negative Numbers

Bulletproof range proofs strictly operate on non-negative integers. Because quantized facial vectors may contain negative values, a mathematical shift is required before generating the proof.

If a value is quantized to k bits, the maximum possible positive value is $M = 2^{k-1} - 1$. To ensure all inputs are strictly positive, we offset the original value v by M:

$$v_{shifted} = v + M$$

For an 8-bit quantization system ($M = 127$), a value of -127 becomes 0, and a value of 127 becomes 254. The resulting $v_{shifted}$ is guaranteed to safely fit within an unsigned 8-bit integer space $[0, 2^8 - 1]$.

8 Python Integration Example

Once compiled, the module can be imported natively into Python. The following script demonstrates the end-to-end proving and verification process utilizing the mathematical shift.

```
import python_bulletproofs

# 1. Setup Parameters
k_bits = 8
M = (1 << (k_bits - 1)) - 1  # 127 for 8-bit

# 2. Example Quantized Value
original_value = -42
shifted_value = original_value + M

# 3. Client Side: Generate Proof
commitment, proof = python_bulletproofs.prove_range(shifted_value, k_bits)

# 4. Server Side: Verify Proof
is_valid = python_bulletproofs.verify_range(commitment, proof, k_bits)

if is_valid:
    print("VERIFIED: The hidden value safely falls within bounds.")
else:
    print("REJECTED: Invalid proof or out of bounds.")
```